Day 9 — Advanced JavaScript

Solved Tasks and Complete Working Example

This solution covers the tasks in the supplied Day 9 PDF: an interactive webpage, button-click and form-submission events, dynamic data, array filtering/searching, and asynchronous operations with API responses.

Included topics

- DOM manipulation and event handling
- Array methods: map, filter, and reduce
- ES6 features, arrow functions, and destructuring
- Spread and rest operators
- Promises, async/await, and Fetch API basics

How to use this answer

Copy the complete HTML code in this document into a file named **index.html**. Open it in a modern browser. The page works without setup; the API button uses the public JSONPlaceholder demo API and needs an internet connection.

1. Complete Solution — index.html

The example is a small student directory. You can add students, search names, filter by course, see the total count, and load sample posts asynchronously from an API.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Day 9 - Advanced JavaScript</title>
  <style>
    * { box-sizing: border-box; }
    body {
      margin: 0; padding: 28px 16px; font-family: Arial, sans-serif;
      background: #f2f6fb; color: #1f2937;
    }
    main {
      max-width: 850px; margin: auto; padding: 24px;
      background: white; border-radius: 14px;
      box-shadow: 0 5px 22px #17365d18;
    }
    h1 { color: #17365d; margin-top: 0; }
    form, .controls { display: flex; gap: 10px; flex-wrap: wrap; margin: 14px 0; }
    input, select, button {
      padding: 10px 12px; border: 1px solid #cbd5e1;
      border-radius: 7px; font-size: 14px;
    }
    input { flex: 1; min-width: 150px; }
    button { background: #24527a; color: white; border: 0; cursor: pointer; }
    button:hover { background: #17365d; }
    .summary { padding: 12px; background: #eaf2fb; border-radius: 8px; }
    .student, .post { border: 1px solid #e2e8f0; padding: 12px; margin: 9px 0; border-radius: 8px; }
    .muted { color: #64748b; font-size: 13px; }
    #status { min-height: 20px; color: #24527a; }
  </style>
</head>
<body>
<main>
  <h1>Student Directory</h1>
  <p>Day 9 practice: DOM, events, arrays, ES6, and asynchronous JavaScript.</p>

  <form id="studentForm">
    <input id="studentName" type="text" placeholder="Student name" required>
    <select id="studentCourse" aria-label="Course">
      <option value="JavaScript">JavaScript</option>
      <option value="Python">Python</option>
      <option value="Java">Java</option>
    </select>
    <button type="submit">Add Student</button>
  </form>

  <div class="controls">
    <input id="searchInput" type="search" placeholder="Search by name...">
    <select id="filterCourse" aria-label="Filter by course">
      <option value="All">All courses</option>
      <option value="JavaScript">JavaScript</option>
      <option value="Python">Python</option>
      <option value="Java">Java</option>
    </select>
    <button id="resetButton" type="button">Reset</button>
  </div>

  <p class="summary" id="summary">Loading students...</p>
  <section id="studentList" aria-live="polite"></section>

  <h2>Asynchronous API Example</h2>
  <p class="muted">Click below to fetch sample posts from JSONPlaceholder.</p>
  <button id="loadPostsButton" type="button">Load API Posts</button>
  <p id="status" role="status"></p>
  <section id="postList"></section>
</main>

<script>
  // ES6: const, let, arrow functions, template literals, and destructuring.
  let students = [
    { id: 1, name: "Aarav", course: "JavaScript" },
    { id: 2, name: "Priya", course: "Python" },
    { id: 3, name: "Rohan", course: "Java" },
```

```
  { id: 4, name: "Neha", course: "JavaScript" }
];

const studentForm = document.querySelector("#studentForm");
const studentName = document.querySelector("#studentName");
const studentCourse = document.querySelector("#studentCourse");
const searchInput = document.querySelector("#searchInput");
const filterCourse = document.querySelector("#filterCourse");
const studentList = document.querySelector("#studentList");
const summary = document.querySelector("#summary");

// DOM manipulation + array filter/map/reduce.
const renderStudents = () => {
  const query = searchInput.value.trim().toLowerCase();
  const course = filterCourse.value;

  const visibleStudents = students
    .filter(({ name, course: studentCourseName }) =>
      name.toLowerCase().includes(query) &&
      (course === "All" || studentCourseName === course)
    );

  studentList.innerHTML = visibleStudents.length
    ? visibleStudents.map(({ name, course }) => `
        <article class="student">
          <strong>${escapeHTML(name)}</strong>
          <span class="muted"> — ${escapeHTML(course)}</span>
        </article>
      `).join("")
    : "<p>No matching students found.</p>";

  const courseCount = students.reduce((total, student) => {
    total[student.course] = (total[student.course] || 0) + 1;
    return total;
  }, {});

  summary.textContent =
    `Showing ${visibleStudents.length} of ${students.length} students. ` +
    `Course totals: ${Object.entries(courseCount)
      .map(([name, count]) => `${name}: ${count}`).join(", ") || "none"}.`;
};

// Small helper so user-entered text is inserted safely as text.
function escapeHTML(value) {
  return value.replace(/[&<>"']/g, character => ({
    "&": "&amp;", "<": "&lt;", ">": "&gt;",
    '"': "&quot;", "'": "&#39;"
  }[character]));
}

// Form-submission event.
studentForm.addEventListener("submit", (event) => {
  event.preventDefault();
  const name = studentName.value.trim();
  if (!name) return;

  const newStudent = {
    id: Date.now(),
    name,
    course: studentCourse.value
  };
  students = [...students, newStudent]; // Spread operator.
  studentForm.reset();
  renderStudents();
});

// Input and change events update the page dynamically.
searchInput.addEventListener("input", renderStudents);
filterCourse.addEventListener("change", renderStudents);

// Button-click event: reset controls.
document.querySelector("#resetButton").addEventListener("click", () => {
  searchInput.value = "";
  filterCourse.value = "All";
  renderStudents();
});

// Async/await + Fetch API + Promise handling.
document.querySelector("#loadPostsButton").addEventListener("click", async () => {
  const status = document.querySelector("#status");
  const postList = document.querySelector("#postList");
```

```
    status.textContent = "Loading posts...";
    postList.innerHTML = "";

    try {
      const response = await fetch(
        "https://jsonplaceholder.typicode.com/posts?_limit=5"
      );
      if (!response.ok) throw new Error(`HTTP error: ${response.status}`);

      const posts = await response.json();
      postList.innerHTML = posts.map(({ title, body }) => `
        <article class="post">
          <strong>${escapeHTML(title)}</strong>
          <p>${escapeHTML(body)}</p>
        </article>
      `).join("");
      status.textContent = `Loaded ${posts.length} posts successfully.`;
    } catch (error) {
      status.textContent = "Could not load posts. Check your internet connection and try again.";
      console.error("API request failed:", error);
    }
  });

  // Rest operator example: collect any number of names into an array.
  const collectNames = (...names) => names;
  console.log("Example rest operator:", ...collectNames("Aarav", "Priya"));

  renderStudents();
</script>
</body>
</html>
```

2. How the solution meets the tasks

Interactive webpage: The student directory updates in the browser without reloading the page.

Button-click and form-submission events: The Reset and Load API Posts buttons use click events; the Add Student form uses a submit event.

Display dynamic data: New student records are added to the students array and the DOM is rendered again.

Array filtering and searching: filter() searches names and applies the selected course; map() creates HTML for visible students.

Array reduce(): reduce() calculates how many students belong to each course.

ES6 features: The code uses const/let, arrow functions, template literals, object destructuring, spread, and rest.

Promises and asynchronous JavaScript: fetch() returns a Promise; async/await waits for the HTTP response and JSON data.

Fetch API and API responses: The page requests five demo posts, checks response.ok, parses JSON, and handles errors.

3. Run the project

• Open Notepad, Visual Studio Code, or another text editor.

• Copy the complete code from Section 1 and save it as index.html.

• Double-click index.html to open it in a modern web browser.

• Add a student, search by name, choose a course, and press Reset to test the DOM and events.

• Press Load API Posts while connected to the internet to test Fetch API and async/await.

4. Expected output

A working Student Directory appears with sample students, a search field, a course filter, a live summary with course totals, and an API section. Adding or filtering students updates the visible list immediately. The API section displays sample post titles and bodies when the request succeeds.

Note: The supplied task sheet lists the required topics and expected outcomes, but does not provide individual question wording. This PDF provides a complete sample solution that covers those listed requirements.